02471 - Problem Set 2

Daniel Borges
s263041

October 25, 2026

Problem 2.1

The standard filtering operation using a Finite Impulse Response (FIR) filter is defined by the discrete convolution sum. The output y_n at time step n is computed as:

$$y_n = \sum_{l=0}^{L-1} \theta_l x_{n-l} \tag{1}$$

where θ_l are the filter coefficients for $l = 0, 1, \ldots, L-1$.

To express this operation as a matrix-vector product $y = X\theta$, we structure the filter coefficients as a column vector and the input signal as a matrix where each row represents the time-delayed samples of the input signal interacting with the filter.

The filter coefficients are arranged into an $L \times 1$ column vector θ:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{L-1} \end{bmatrix} \tag{2}$$

To satisfy the convolution sum, each row of the matrix X must contain the current and past values of the signal x_n, shifted by one time step per column. Assuming we compute the valid convolution where we only compute outputs for time indices where the filter fully overlaps with the available data (i.e., $n \in [L-1, N-1]$), X is constructed as an $(N-L+1) \times L$ matrix:

$$X = \begin{bmatrix} x_{L-1} & x_{L-2} & \ldots & x_0 \\ x_L & x_{L-1} & \ldots & x_1 \\ x_{L+1} & x_L & \ldots & x_2 \\ \vdots & \vdots & \ddots & \vdots \\ x_{N-1} & x_{N-2} & \ldots & x_{N-L} \end{bmatrix} \tag{3}$$

If we assume a full convolution where zero-padding is used for x_n outside the range $0 \leq n < N$, the matrix X would be expanded with zeros at the boundaries, resulting in an $(N+L-1) \times L$ matrix:

$$X_{full} = \begin{bmatrix} x_0 & 0 & \ldots & 0 \\ x_1 & x_0 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ x_{N-1} & x_{N-2} & \ldots & x_{N-L} \\ 0 & x_{N-1} & \ldots & x_{N-L+1} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & x_{N-1} \end{bmatrix} \tag{4}$$

With X and θ defined, the filtering operation is executed via matrix multiplication, yielding the output column vector y:

$$y = X\theta = \begin{bmatrix} y_{L-1} \\ y_L \\ \vdots \\ y_{N-1} \end{bmatrix} \tag{5}$$

Each element of y corresponds exactly to the dot product of a row in X and the vector θ, thus flawlessly replicating the convolution sum.

Problem 2.2

We begin with the definition of the Mean Squared Error (MSE) and add/subtract the expected value of the estimator, $\mathbb{E}[\hat{\theta}]$, inside the square:

$$\begin{aligned} \text{MSE} &= \mathbb{E}\left[(\hat{\theta} - \theta_o)^2\right] \\ &= \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}] + \mathbb{E}[\hat{\theta}] - \theta_o)^2\right] \end{aligned} \tag{6}$$

Expanding the square yields three terms:

$$\text{MSE} = \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2\right] + \mathbb{E}\left[(\mathbb{E}[\hat{\theta}] - \theta_o)^2\right] + 2\mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])(\mathbb{E}[\hat{\theta}] - \theta_o)\right] \tag{7}$$

The first term is the standard definition of the estimator's variance: $\text{Var}(\hat{\theta}) = \mathbb{E}\left[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2\right]$. The second term consists of non-random constants, so $\mathbb{E}\left[(\mathbb{E}[\hat{\theta}] - \theta_o)^2\right] = (\mathbb{E}[\hat{\theta}] - \theta_o)^2 = \text{Bias}(\hat{\theta})^2$. For the cross-term, the term $(\mathbb{E}[\hat{\theta}] - \theta_o)$ is a constant with respect to the outer expectation and can be pulled out: $2(\mathbb{E}[\hat{\theta}] - \theta_o) \cdot \mathbb{E}[\hat{\theta} - \mathbb{E}[\hat{\theta}]]$. Because $\mathbb{E}[\hat{\theta} - \mathbb{E}[\hat{\theta}]] = \mathbb{E}[\hat{\theta}] - \mathbb{E}[\hat{\theta}] = 0$, the entire cross-term evaluates to zero.

Summing these parts gives the standard decomposition:

$$\text{MSE} = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2 \tag{8}$$

An unbiased estimator inherently satisfies $\text{Bias}(\hat{\theta}) = 0$, reducing its MSE entirely to the estimator's variance: $\text{MSE}_{unbiased} = \text{Var}(\hat{\theta})$.

A biased estimator is often better suited for minimizing the overall MSE. Introducing a small, controlled amount of bias can act as a regularization mechanism. This restricts the model's sensitivity to noisy data, which can reduce its variance. If the reduction in variance exceeds the penalty of the squared bias introduced, the overall MSE will decrease:

$$\text{Var}_{biased}(\hat{\theta}) + \text{Bias}_{biased}(\hat{\theta})^2 < \text{Var}_{unbiased}(\hat{\theta}) \tag{9}$$

Because the MSE is a sum of strictly non-negative terms, allowing a non-zero bias lets us exploit this bias-variance tradeoff to achieve a lower total error than would be mathematically possible if we strictly enforced $\text{Bias}(\hat{\theta}) = 0$.

Problem 2.3

A function f defined on a convex set $\mathcal{C}$ is convex if, for all $\theta_1, \theta_2 \in \mathcal{C}$ and all $\lambda \in [0, 1]$,

$$f\big(\lambda\theta_1 + (1-\lambda)\theta_2\big) \leq \lambda f(\theta_1) + (1-\lambda) f(\theta_2). \tag{10}$$

The domain $\mathbb{R}^l$ of a norm is a convex set. Let $\|\cdot\|_p$ satisfy the three norm properties and take $\theta_1, \theta_2 \in \mathbb{R}^l$ and $\lambda \in [0,1]$. Then

$$\begin{aligned}\|\lambda\theta_1 + (1-\lambda)\theta_2\|_p &\leq \|\lambda\theta_1\|_p + \|(1-\lambda)\theta_2\|_p \\ &= |\lambda|\,\|\theta_1\|_p + |1-\lambda|\,\|\theta_2\|_p \\ &= \lambda\|\theta_1\|_p + (1-\lambda)\|\theta_2\|_p\end{aligned}$$

This is exactly the definition of convexity, so every norm is convex.

Problem 2.4

The autocorrelation at time instants n, m is defined as $r(n,m) = \mathbb{E}[\mathrm{u}_n \mathrm{u}_m]$ (eq. 2.101). For a wide-sense stationary process it only depends on the lag $k = n-m$, so we write $r(k) = \mathbb{E}[\mathrm{u}_n \mathrm{u}_{n-k}]$.

Since $|a| < 1$ the process is wide-sense stationary. The noise v_n is white and x_{n-k} depends only on v_m with $m \leq n-k$. Hence $\mathbb{E}[\mathrm{v}_n \mathrm{x}_{n-k}] = 0$ for $k \geq 1$, and $\mathbb{E}[\mathrm{v}_n \mathrm{x}_n] = \sigma_v^2$.

For $k = 0$:

$$r_x(0) = \mathbb{E}[\mathrm{x}_n(a\mathrm{x}_{n-1} + \mathrm{v}_n)] = a\, r_x(1) + \sigma_v^2. \tag{11}$$

For $k \geq 1$:

$$r_x(k) = \mathbb{E}[(a\mathrm{x}_{n-1} + \mathrm{v}_n)\mathrm{x}_{n-k}] = a\, r_x(k-1) \quad \Rightarrow \quad r_x(k) = a^k r_x(0). \tag{12}$$

Inserting $r_x(1) = a\, r_x(0)$ in the first equation gives $r_x(0) = a^2 r_x(0) + \sigma_v^2$, which means:

$$r_x(0) = \frac{\sigma_v^2}{1-a^2}. \tag{13}$$

Because $r_x(-k) = r_x(k)$, we obtain for all k:

$$r_x(k) = \frac{\sigma_v^2}{1-a^2}\, a^{|k|}. \tag{14}$$

Using $\mathbb{E}[\mathrm{x}_n] = 0$, we find the autocorrelation of y_n with $m = n-k$:

$$\begin{aligned}r_y(n, n-k) &= \mathbb{E}[(\mathrm{x}_n + b)(\mathrm{x}_{n-k} + b)] = r_x(k) + b\,\mathbb{E}[\mathrm{x}_n] + b\,\mathbb{E}[\mathrm{x}_{n-k}] + b^2 \\ &= \frac{\sigma_v^2}{1-a^2}\, a^{|k|} + b^2.\end{aligned} \tag{15}$$

The result only depends on the lag k and $\mathbb{E}[\mathrm{y}_n] = b$ is constant, so y_n is wide-sense stationary and $r_y(n, n-k) = r_y(k)$. The offset adds the constant b^2 at every lag. For $a = 0.8$, $b = 0.5$, and $\sigma_v^2 = 1$:

$$r_y(-2) = r_y(n, n+2) = \frac{1}{1-0.64}\, 0.8^2 + 0.25 = \frac{0.64}{0.36} + 0.25 = \frac{16}{9} + \frac{1}{4} = \frac{73}{36} \approx 2.03. \tag{16}$$

Problem 2.5

In a filter setup, the filter acts on the input x_n and produces the output $\hat{d}_n = \sum_{l=0}^{L-1} \theta_l x_{n-l} = \theta^\top \boldsymbol{x}_n$. The desired signal d_n is the signal we would like the output to reproduce, and the error signal is the difference:

$$e_n = d_n - \hat{d}_n = d_n - \theta^\top \boldsymbol{x}_n. \tag{17}$$

The filter is chosen to minimize $\mathbb{E}[e_n^2]$. In this problem, the input is $x_n = s_n + w_n$, the desired signal is the clean signal $d_n = s_n$, and the error is $e_n = s_n - \hat{s}_n$, which is the remaining noise plus the distortion of the signal.

Setting the gradient of $J(\theta) = \mathbb{E}[(d_n - \theta^\top \boldsymbol{x}_n)^2]$ to zero gives the normal equations:

$$\Sigma_x \, \theta_* = p, \qquad [\Sigma_x]_{ij} = r_x(i-j), \qquad p_k = \mathbb{E}[d_n x_{n-k}] = r_{dx}(k). \tag{18}$$

Since s_n and w_n are uncorrelated, $\Sigma_x = \Sigma_s + \Sigma_w$ and $p_k = r_s(k)$. In the frequency domain, for $L \to \infty$, the solution is:

$$H(f) = \frac{S_s(f)}{S_s(f) + S_w(f)}, \tag{19}$$

which passes frequencies where the signal dominates and attenuates those where the noise dominates.

The filter must be long enough to resolve the spectral structure of $S_s/(S_s+S_w)$ since a longer filter has a finer frequency resolution and approximates the ideal response better, meaning the MSE decreases with L. However, Σ_x and p are estimated from a finite number of samples, and the number of parameters grows with L. Estimating the filter on the first half of the data and measuring the MSE on the second half gives a chosen length of $L = 39$ (Figure 1).

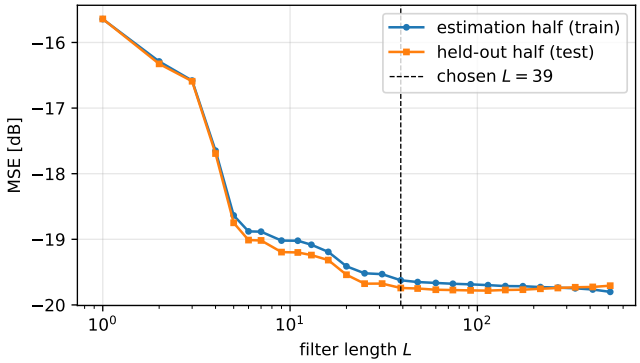


Figure 1: MSE as a function of the filter length L, on the data used for the estimation and on held-out data.

Using all samples to estimate the filter, the signal-to-noise ratio is improved from 3.2 dB at the input to 8.9 dB at the output, a gain of 5.7 dB. Figure 2 shows the frequency response of the Wiener filter together with the ideal response. The filter approximates the ideal response with a gain close to 0 dB where the signal spectrum lies above the noise spectrum, and strong attenuation where the noise dominates.

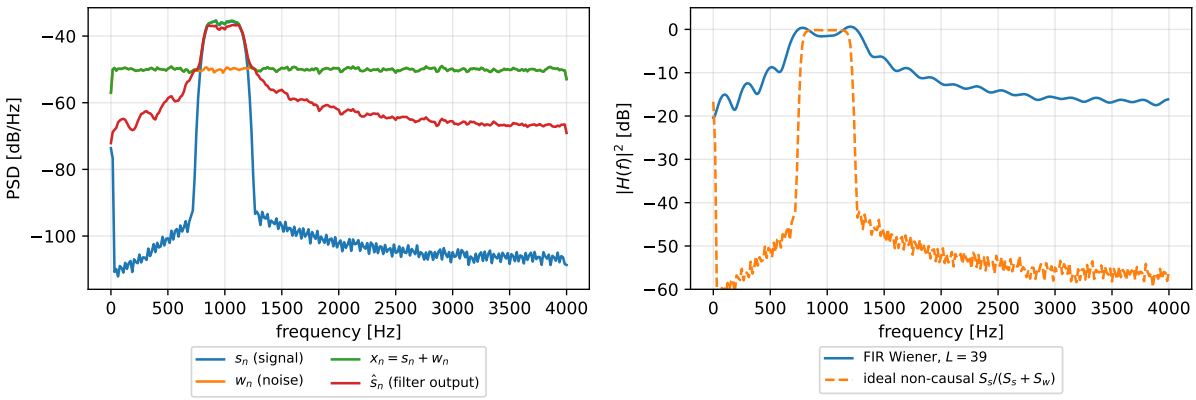


Figure 2: Left: spectra of the signal, noise, input and filter output. Right: frequency response of the FIR Wiener filter and the ideal response.

Problem 2.6

The signal is sampled at $f_s = 22050$ Hz and lasts 4.04 s. The short-time Fourier transform is subject to a time and frequency resolution trade-off where a window of N_w samples gives a frequency resolution of about f_s/N_w and a time resolution of about N_w/f_s. A window length of 1024 samples gives a resolution of about 21.5 Hz, which is fine enough to separate the low-frequency drum from drums with energy at higher frequencies. The window is short compared to the time between the beats so successive beats do not smear into each other. A Hann window is used because it has low side lobes so the strong low-frequency components do not leak into the weak high-frequency content. An overlap of 87.5% is used to avoid losing information at the window edges and to give a smooth time axis and precise localization of the beats.

The beats were found as peaks of the spectral flux (Figure 3), detecting 17 beats with a median time between consecutive beats of 0.238 s, corresponding to a tempo of about 252 beats per minute. The 87.5% overlap gives a time resolution of 5.8 ms in the localization of the beats.

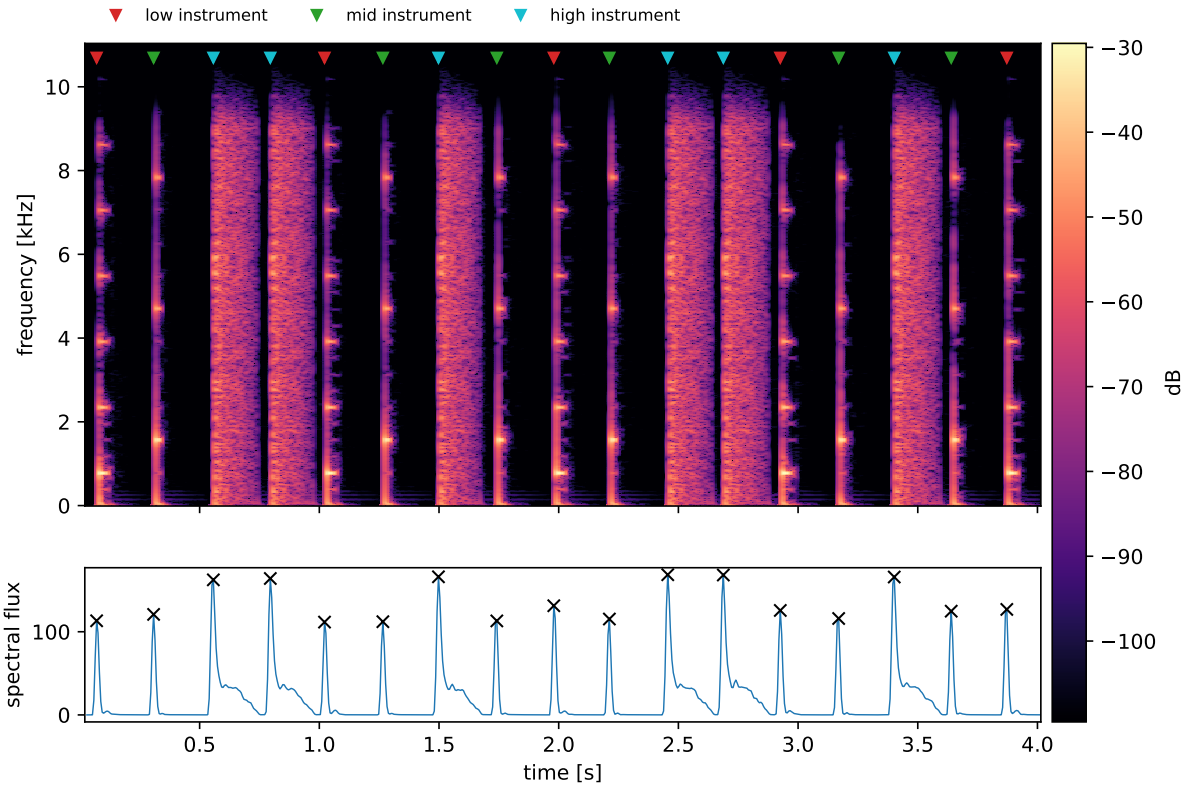


Figure 3: Spectrogram (Hann window, 1024 samples, 87.5 % overlap) with detected beats and the spectral-flux onset function.

The beats were grouped into three classes using k-means (Figure 4). The three instruments are distinguished by where their energy is located (Table 1): the class with the lowest centroid corresponds to a bass-drum type sound, the middle class to a snare type sound, and the class with the highest centroid to a hi-hat type sound.

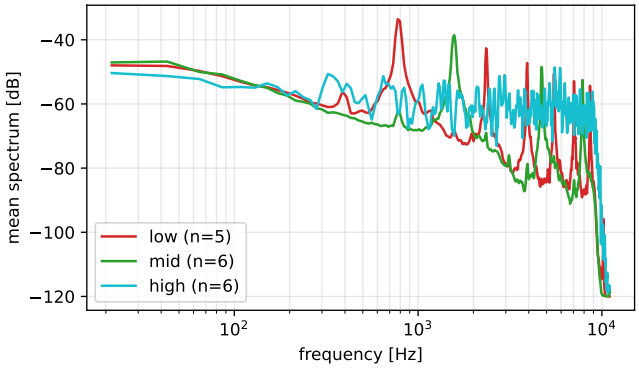


Figure 4: Mean spectrum of the beats in each class.

Class	Number of beats	Spectral centroid [Hz]	Peak frequency [Hz]
low	5	2253	775
mid	6	2544	1572
high	6	4179	5534

Table 1: Spectral signature of the instrument classes.

Problem 2.7

The signals in the setup are the source signal which is the speech s_n that we want to recover, the unknown linear system h that the noise passes through, the primary microphone signal $p_n = s_n + (h * v)_n$ which is the desired signal, the reference microphone signal v_n which is the input x_n, the adaptive FIR filter w, the filter output a_n which is an estimate of the noise component, the error signal $e_n = p_n - a_n$ which is the output of the whole system, and the update of the filter coefficients c.

The adaptive algorithm minimizes the power of the error $e_n = p_n - a_n$:

$$\mathbb{E}[e_n^2] = \mathbb{E}\big[(s_n + n_n - a_n)^2\big] = \mathbb{E}[s_n^2] + \mathbb{E}\big[(n_n - a_n)^2\big] + 2\,\mathbb{E}\big[s_n(n_n - a_n)\big]. \tag{20}$$

If the speech is uncorrelated with the noise, the cross-term vanishes and $\mathbb{E}[e_n^2] = \mathbb{E}[s_n^2] + \mathbb{E}\big[(n_n - a_n)^2\big]$. Minimizing the output power is the same as minimizing the noise residual $\mathbb{E}[(n_n - a_n)^2]$, which is minimized for $w = h$, and then $e_n = s_n$. If the speech is correlated with the reference, the filter reduces the output power by also cancelling parts of the speech itself and the output is then a distorted speech signal.

The speech signal was scaled to a peak value of 0.5, and the reference noise v_n is white Gaussian noise filtered through low-pass, high-pass, and band-pass impulse responses. The adaptive filters are LMS, NLMS, and RLS. RLS converges in the order of M to $2M$ samples independently of the eigenvalue spread but at the cost of $O(M^2)$ operations per sample. LMS and NLMS cost $O(M)$ and converge more slowly, while NLMS removes the dependence on the input power through normalization so it is easier to tune (Figure 5).

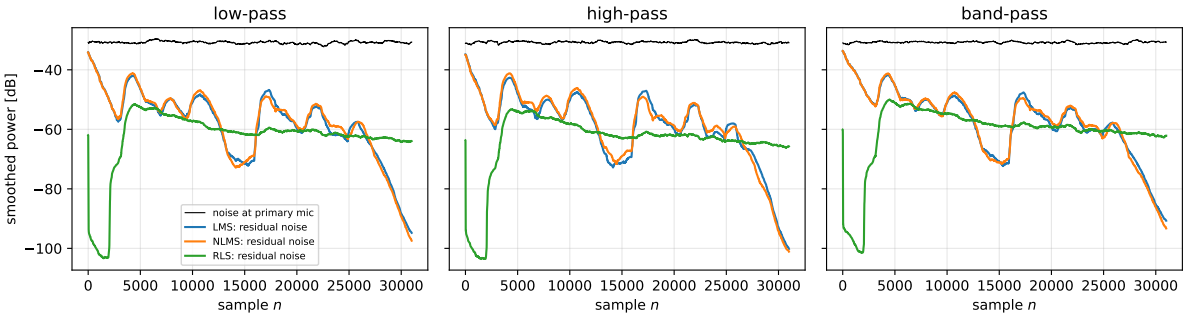


Figure 5: Learning curves: smoothed power of the noise at the primary microphone and of the residual noise for the three algorithms.

A large step size gives fast convergence and tracking but a large excess error, while a small step size gives low misadjustment but slow convergence (Table 2 and Figure 6).

Noise path	Algorithm	Parameter	SNR in [dB]	SNR out [dB]	Misalignment [dB]
low-pass	LMS	0.153	0.0	23.6	-70.3
low-pass	NLMS	0.01	0.0	24.0	-73.9
low-pass	RLS	1	0.0	29.3	-33.5
high-pass	LMS	0.229	-0.0	23.9	-77.9
high-pass	NLMS	0.01	-0.0	24.6	-76.2
high-pass	RLS	1	-0.0	30.8	-35.1
band-pass	LMS	0.201	0.0	24.0	-64.6
band-pass	NLMS	0.01	0.0	24.2	-67.5
band-pass	RLS	1	0.0	27.7	-31.9

Table 2: Performance of the adaptive noise cancellers.

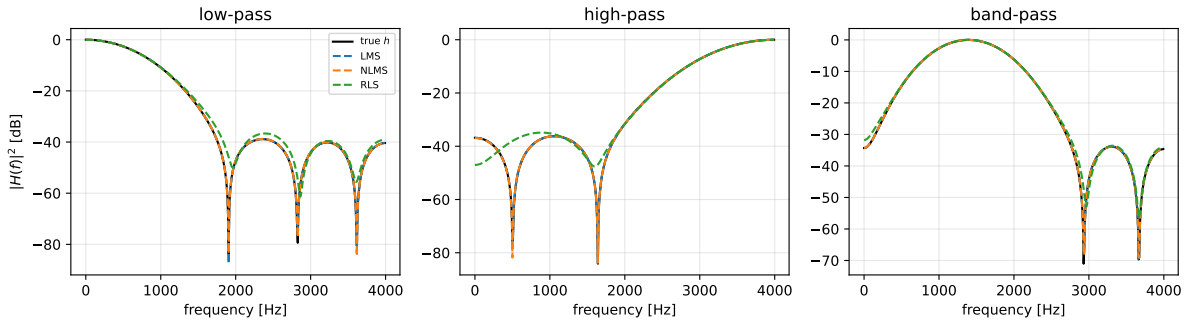


Figure 6: Frequency response of the true noise path and of the estimated adaptive filters.

Adding a fraction of the speech to the reference signal causes the performance to degrade because the filter learns to cancel part of the speech (Table 3), confirming why the source and noise must be uncorrelated.

α	SNR out [dB]
0.0	4.5
0.1	3.4
0.3	1.4

Table 3: NLMS output SNR when a fraction α of the speech leaks into the reference microphone.

Appendix: Python code for Problems 2.5–2.7

```
"""
02471 Machine Learning for Signal Processing -- Problem set 2, problems 2.5-2.7.

Usage:    python solutions.py [data_dir] [out_dir]
          data_dir (default ./data) must contain
              problem2_5_signal.mat, problem2_5_noise.mat,
              problem2_6.wav,
              problem2_7_speech.mat, problem2_7_lpir.mat,
              problem2_7_hpir.mat, problem2_7_bpir.mat
Output:   out_dir/figs/*.pdf   figures used by exercise_solutions.tex
          out_dir/results.tex   numbers used by exercise_solutions.tex
          out_dir/audio/*.wav   noisy / cleaned speech for the listening test
"""
import sys
from pathlib import Path

import numpy as np
import scipy.io as sio
import scipy.signal as sg
from scipy.cluster.vq import kmeans2
from scipy.io import wavfile
from scipy.linalg import solve_toeplitz

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BASE = Path(__file__).resolve().parent
DATA = Path(sys.argv[1]) if len(sys.argv) > 1 else BASE / "data"
OUT = Path(sys.argv[2]) if len(sys.argv) > 2 else BASE
FIGS, AUDIO = OUT / "figs", OUT / "audio"
FIGS.mkdir(parents=True, exist_ok=True)
AUDIO.mkdir(parents=True, exist_ok=True)
FS = 8000
RNG = np.random.default_rng(0)
R = []  # lines of results.tex


def macro(name, value):
    R.append(f"\\newcommand{{\\{name}}}{{{value}}}")


def load_mat(name):
    """Load the (single) numeric variable of a .mat file as a 1-D float array."""
    m = sio.loadmat(DATA / name)
    arrs = [np.asarray(v) for k, v in m.items()
            if not k.startswith("__") and np.issubdtype(np.asarray(v).dtype, np.number)]
    return max(arrs, key=lambda a: a.size).astype(float).ravel()


def db(x):
    return 10 * np.log10(x)


# ---------------------------------------------------------------------------
# Problem 2.5  FIR Wiener filter
# ---------------------------------------------------------------------------
def xcorr(a, b, L):
    """Biased estimate of E[a_n b_{n-k}], k = 0..L-1."""
    N = len(a)
    return np.array([a[k:] @ b[:N - k] for k in range(L)]) / N


def wiener_fir(x, d, L):
    """Solve Sigma_x theta = p, Sigma_x = Toeplitz(r_x), p_k = E[d_n x_{n-k}]."""
    return solve_toeplitz(xcorr(x, x, L), xcorr(d, x, L))


def problem_2_5():
    s, w = load_mat("problem2_5_signal.mat"), load_mat("problem2_5_noise.mat")
    n = min(len(s), len(w))
```

```
    s, w = s[:n], w[:n]
    x = s + w
    half = n // 2                       # estimate on first half, test on second
    Ls = np.unique(np.round(np.geomspace(1, 512, 30)).astype(int))
    mse_tr, mse_te = [], []
    for L in Ls:
        th = wiener_fir(x[:half], s[:half], L)
        sh = sg.lfilter(th, 1, x)
        mse_tr.append(np.mean((s[:half] - sh[:half]) ** 2))
        mse_te.append(np.mean((s[half:] - sh[half:]) ** 2))
    mse_tr, mse_te = np.array(mse_tr), np.array(mse_te)
    # smallest L whose held-out MSE is within 2 % of the best held-out MSE
    L_opt = int(Ls[np.argmax(mse_te <= 1.02 * mse_te.min())])

    th = wiener_fir(x, s, L_opt)        # final filter on all data
    sh = sg.lfilter(th, 1, x)
    snr_in = db(np.sum(s ** 2) / np.sum(w ** 2))
    snr_out = db(np.sum(s ** 2) / np.sum((s - sh) ** 2))

    fig, ax = plt.subplots(figsize=(5.5, 3.3))
    ax.semilogx(Ls, db(mse_tr), "o-", ms=3, label="estimation␣half␣(train)")
    ax.semilogx(Ls, db(mse_te), "s-", ms=3, label="held-out␣half␣(test)")
    ax.axvline(L_opt, color="k", ls="--", lw=.8, label=f"chosen␣$L={L_opt}$")
    ax.set_xlabel("filter␣length␣$L$"); ax.set_ylabel("MSE␣[dB]"); ax.legend()
    ax.grid(alpha=.3); fig.tight_layout(); fig.savefig(FIGS / "p25_mse_vs_L.pdf")

    nper = 512
    f, Ps = sg.welch(s, FS, nperseg=nper); _, Pw = sg.welch(w, FS, nperseg=nper)
    _, Px = sg.welch(x, FS, nperseg=nper); _, Pe = sg.welch(sh, FS, nperseg=nper)
    fh, H = sg.freqz(th, 1, worN=512, fs=FS)
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
    for P, lab in [(Ps, "$s_n$␣(signal)"), (Pw, "$w_n$␣(noise)"),
                   (Px, "$x_n=s_n+w_n$"), (Pe, r"$\hat␣s_n$␣(filter␣output)")]:
        ax[0].plot(f, db(P), label=lab)
    ax[0].set_xlabel("frequency␣[Hz]"); ax[0].set_ylabel("PSD␣[dB/Hz]")
    ax[0].legend(fontsize=8, loc="upper␣center", bbox_to_anchor=(0.5, -0.18), ncol=2)
    ax[0].grid(alpha=.3)
    ax[1].plot(fh, db(np.abs(H) ** 2 + 1e-12), label=f"FIR␣Wiener,␣$L={L_opt}$")
    ax[1].plot(f, db(Ps / (Ps + Pw)), "--", label=r"ideal␣non-causal␣$S_s/(S_s+S_w)$")
    ax[1].set_ylim(-60, 5)
    ax[1].set_xlabel("frequency␣[Hz]"); ax[1].set_ylabel(r"$|H(f)|^2$␣[dB]")
    ax[1].legend(fontsize=8, loc="upper␣center", bbox_to_anchor=(0.5, -0.18), ncol=1)
    ax[1].grid(alpha=.3)
    fig.tight_layout(); fig.savefig(FIGS / "p25_freq.pdf", bbox_inches="tight")

    macro("resWienerL", L_opt)
    macro("resWienerSNRin", f"{snr_in:.1f}")
    macro("resWienerSNRout", f"{snr_out:.1f}")
    macro("resWienerGain", f"{snr_out␣-␣snr_in:.1f}")
    macro("resWienerLtestMSE", f"{db(mse_te[Ls␣==␣L_opt][0]):.1f}")
    macro("resWienerMinMSE", f"{db(mse_te.min()):.1f}")
    macro("resWienerLmin", int(Ls[np.argmin(mse_te)]))


# ---------------------------------------------------------------------------
# Problem 2.6  Spectrogram of audio
# ---------------------------------------------------------------------------
def problem_2_6():
    fs, y = wavfile.read(DATA / "problem2_6.wav")
    y = y.astype(float)
    if y.ndim > 1:
        y = y.mean(axis=1)
    y /= np.max(np.abs(y))
    nper, hop = 1024, 128
    f, t, S = sg.spectrogram(y, fs, window="hann", nperseg=nper,
                             noverlap=nper - hop, mode="magnitude")
    SdB = 20 * np.log10(S + 1e-10)

    # onset detection: spectral flux of log-compressed magnitude
    C = np.log1p(100 * S / S.max())
    flux = np.maximum(np.diff(C, axis=1), 0).sum(axis=0)
    flux = np.convolve(flux, np.hanning(5) / np.hanning(5).sum(), "same")
    min_dist = int(0.06 * fs / hop)
```

```
    pk, _ = sg.find_peaks(flux, height=flux.mean() + 1.0 * flux.std(), distance=min_dist)
    onsets = t[pk + 1]
    ioi = np.diff(onsets)

    # per-onset spectral signature: mean log-spectrum in 12 log-spaced bands
    edges = np.geomspace(40, fs / 2, 13)
    feats, spec = [], []
    for i in pk + 1:
        seg = S[:, i:i + 8].mean(axis=1)
        spec.append(seg)
        b = [seg[(f >= lo) & (f < hi)].sum() ** 2 for lo, hi in zip(edges[:-1], edges
            [1:])]
        b = db(np.array(b) + 1e-12)
        feats.append(b - b.max())
    feats, spec = np.array(feats), np.array(spec)
    K = 3
    _, lab = kmeans2(feats, K, minit="++", seed=0)
    cent = np.array([np.sum(f * spec[lab == k].mean(0)) / np.sum(spec[lab == k].mean(0))
                     if np.any(lab == k) else np.nan for k in range(K)])
    order = np.argsort(cent)                        # low -> high
    rank = {int(k): r for r, k in enumerate(order)}
    lab = np.array([rank[int(l)] for l in lab])
    names = ["low", "mid", "high"]
    cols = ["tab:red", "tab:green", "tab:cyan"]

    fig, ax = plt.subplots(2, 1, figsize=(10, 6), sharex=True,
                           gridspec_kw={"height_ratios": [3, 1]})
    im = ax[0].pcolormesh(t, f / 1e3, SdB, shading="auto", cmap="magma",
                          vmin=SdB.max() - 80, vmax=SdB.max())
    for r in range(K):
        ax[0].plot(onsets[lab == r], np.full((lab == r).sum(), f[-1] / 1e3 * .97), "v",
                   color=cols[r], ms=5, label=f"{names[r]} instrument")
    ax[0].set_ylabel("frequency [kHz]"); ax[0].legend(loc="lower left", bbox_to_anchor
        =(0, 1.01), fontsize=8, ncol=3, frameon=False)
    fig.colorbar(im, ax=ax, label="dB", pad=0.01)
    ax[1].plot(t[1:], flux, lw=.7); ax[1].plot(onsets, flux[pk], "kx")
    ax[1].set_xlabel("time [s]"); ax[1].set_ylabel("spectral flux")
    fig.savefig(FIGS / "p26_spectrogram.pdf", bbox_inches="tight")

    fig, ax = plt.subplots(figsize=(5.5, 3.3))
    for r in range(K):
        if np.any(lab == r):
            ax.semilogx(f[1:], db(spec[lab == r].mean(0)[1:] ** 2 + 1e-12),
                        color=cols[r], label=f"{names[r]} (n={np.sum(lab == r)})")
    ax.set_xlabel("frequency [Hz]"); ax.set_ylabel("mean spectrum [dB]")
    ax.grid(alpha=.3, which="both"); ax.legend(); fig.tight_layout()
    fig.savefig(FIGS / "p26_signatures.pdf")

    rows = []
    for r in range(K):
        m = lab == r
        if not m.any():
            continue
        ms = spec[m].mean(0)
        rows.append(f"{names[r]} & {m.sum()} & {np.sum(f * ms) / np.sum(ms):.0f} & "
                    f"{f[1:][np.argmax(ms[1:])]:.0f} \\\\")
    med = np.median(ioi)
    macro("resSixFs", fs)
    macro("resSixDur", f"{len(y) / fs:.2f}")
    macro("resSixDf", f"{fs / nper:.1f}")
    macro("resSixWinMs", f"{1e3 * nper / fs:.0f}")
    macro("resSixHopMs", f"{1e3 * hop / fs:.1f}")
    macro("resSixNons", len(onsets))
    macro("resSixIOI", f"{med:.3f}")
    macro("resSixBPM", f"{60 / med:.0f}")
    macro("resSixRows", " ".join(rows))


# ---------------------------------------------------------------------------
# Problem 2.7  Adaptive noise cancelling: LMS, NLMS, RLS
# ---------------------------------------------------------------------------
def regressors(d, M):
    """Row n = [d_n, d_{n-1}, ..., d_{n-M+1}] (zero padded)."""
```

```
    dp = np.concatenate([np.zeros(M - 1), d])
    return np.lib.stride_tricks.sliding_window_view(dp, M)[:, ::-1]


def lms(d, p, M, mu):
    U, w, e = regressors(d, M), np.zeros(M), np.zeros(len(d))
    for n in range(len(d)):
        e[n] = p[n] - w @ U[n]
        w += mu * e[n] * U[n]
    return e, w


def nlms(d, p, M, mu, eps=1e-6):
    U, w, e = regressors(d, M), np.zeros(M), np.zeros(len(d))
    for n in range(len(d)):
        e[n] = p[n] - w @ U[n]
        w += mu * e[n] * U[n] / (eps + U[n] @ U[n])
    return e, w


def rls(d, p, M, lam, delta=0.01):
    U, w, e = regressors(d, M), np.zeros(M), np.zeros(len(d))
    P = np.eye(M) / (delta * np.var(d))
    for n in range(len(d)):
        u = U[n]
        Pu = P @ u
        k = Pu / (lam + u @ Pu)
        e[n] = p[n] - w @ u
        w += k * e[n]
        P = (P - np.outer(k, Pu)) / lam
    return e, w


def snr(s, resid):
    return db(np.sum(s ** 2) / np.sum(resid ** 2))


def smooth_db(x, N=1000):
    return db(np.convolve(x ** 2, np.ones(N) / N, "valid") + 1e-14)


def problem_2_7():
    s = load_mat("problem2_7_speech.mat")
    s = s / np.max(np.abs(s)) * 0.5
    N = len(s)
    irs = {k: load_mat(f"problem2_7_{k}ir.mat") for k in ("lp", "hp", "bp")}
    names = {"lp": "low-pass", "hp": "high-pass", "bp": "band-pass"}
    SNR_IN = 0.0                                        # dB at the primary microphone
    v = RNG.standard_normal(N)                          # reference noise (unit variance)
    results, curves = [], {}
    rows = []
    sweep_rows = {}

    for key, h in irs.items():
        noise = sg.lfilter(h, 1, v)
        g = np.sqrt(np.sum(s ** 2) / np.sum(noise ** 2) * 10 ** (-SNR_IN / 10))
        d = g * v                                       # reference microphone
        nz = g * noise                                  # noise at primary microphone
        p = s + nz                                      # primary microphone
        M = len(h)
        wtrue = h                                       # noise at primary = h * d, so the
            ideal w is h
        s2d = np.var(d)
        wavfile.write(AUDIO / f"noisy_{key}.wav", FS, (p / np.abs(p).max() * .9 * 32767).
            astype(np.int16))

        # step-size / forgetting-factor sweeps (final SNR over the last half)
        def final_snr(e):
            return snr(s[N // 2:], (e - s)[N // 2:])
        mu_l = np.array([.01, .03, .1, .3, .6]) * 2 / (M * s2d) * 0.5
        mu_n = np.array([.01, .02, .05, .1, .25, .5, 1.0])
        lam_r = np.array([.99, .995, .999, .9995, 1.0])
        sw_l = [final_snr(lms(d, p, M, m)[0]) for m in mu_l]
```

```
        sw_n = [final_snr(nlms(d, p, M, m)[0]) for m in mu_n]
        sw_r = [final_snr(rls(d, p, M, l)[0]) for l in lam_r]
        best = dict(LMS=(mu_l[int(np.argmax(sw_l))], lms),
                    NLMS=(mu_n[int(np.argmax(sw_n))], nlms),
                    RLS=(lam_r[int(np.argmax(sw_r))], rls))
        sweep_rows[key] = (mu_l, sw_l, mu_n, sw_n, lam_r, sw_r)

        curves[key] = {}
        for alg, (par, fn) in best.items():
            e, w = fn(d, p, M, par)
            curves[key][alg] = (e, w)
            mis = db(np.sum((w - wtrue) ** 2) / np.sum(wtrue ** 2))
            rows.append(f"{names[key]} & {alg} & {par:.3g} & {snr(s, p - s):.1f} & "
                        f"{snr(s[N//2:], (e - s)[N//2:]):.1f} & {mis:.1f} \\\\")
            wavfile.write(AUDIO / f"clean_{key}_{alg}.wav", FS,
                          (e / np.abs(p).max() * .9 * 32767).astype(np.int16))
        curves[key]["_noise"] = nz
        curves[key]["_p"] = p
        curves[key]["_d"] = d
        curves[key]["_h"] = wtrue

    # --- figures ------------------------------------------------------------
    fig, ax = plt.subplots(1, 3, figsize=(12, 3.4), sharey=True)
    for a, key in zip(ax, irs):
        nz = curves[key]["_noise"]
        a.plot(smooth_db(nz), "k", lw=.8, label="noise at primary mic")
        for alg in ("LMS", "NLMS", "RLS"):
            e = curves[key][alg][0]
            a.plot(smooth_db(e - s), label=f"{alg}: residual noise")
        a.set_title(names[key]); a.set_xlabel("sample $n$"); a.grid(alpha=.3)
    ax[0].set_ylabel("smoothed power [dB]"); ax[0].legend(fontsize=7)
    fig.tight_layout(); fig.savefig(FIGS / "p27_learning.pdf")

    fig, ax = plt.subplots(1, 3, figsize=(12, 3.4))
    for a, key in zip(ax, irs):
        M = len(irs[key])
        fh, Ht = sg.freqz(curves[key]["_h"], 1, worN=512, fs=FS)
        a.plot(fh, db(np.abs(Ht) ** 2 + 1e-12), "k", lw=1.5, label="true $h$")
        for alg in ("LMS", "NLMS", "RLS"):
            fw, Hw = sg.freqz(curves[key][alg][1], 1, worN=512, fs=FS)
            a.plot(fw, db(np.abs(Hw) ** 2 + 1e-12), "--", label=alg)
        a.set_title(names[key]); a.set_xlabel("frequency [Hz]"); a.grid(alpha=.3)
    ax[0].set_ylabel(r"$|H(f)|^2$ [dB]"); ax[0].legend(fontsize=7)
    fig.tight_layout(); fig.savefig(FIGS / "p27_filters.pdf")

    fig, ax = plt.subplots(1, 3, figsize=(12, 3.2))
    key0 = list(irs)[0]
    mu_l, sw_l, mu_n, sw_n, lam_r, sw_r = sweep_rows[key0]
    ax[0].semilogx(mu_l, sw_l, "o-"); ax[0].set_xlabel(r"LMS $\mu$")
    ax[1].semilogx(mu_n, sw_n, "o-"); ax[1].set_xlabel(r"NLMS $\tilde\mu$")
    ax[2].plot(lam_r, sw_r, "o-"); ax[2].set_xlabel(r"RLS $\lambda$")
    ax[0].set_ylabel("output SNR [dB]")
    for a in ax:
        a.grid(alpha=.3)
    fig.suptitle(f"Parameter sweep, {names[key0]} noise path", fontsize=9)
    fig.tight_layout(); fig.savefig(FIGS / "p27_sweep.pdf")

    # --- experiment: reference microphone picks up the speech ---------------
    key = "bp"
    d, p, h = curves[key]["_d"], curves[key]["_p"], curves[key]["_h"]
    M = len(irs[key])
    leak = []
    for alpha in (0.0, 0.1, 0.3):
        dd = d + alpha * s * np.std(d) / np.std(s)
        e, _ = nlms(dd, p, M, 0.5)
        leak.append((alpha, snr(s[N // 2:], (e - s)[N // 2:])))
    macro("resLeakRows", " ".join(f"{a:.1f} & {x:.1f} \\\\" for a, x in leak))

    macro("resSevenN", N)
    macro("resSevenM", ", ".join(f"{names[k]}: {len(h)}" for k, h in irs.items()))
    macro("resSevenSNRin", f"{SNR_IN:.0f}")
    macro("resSevenRows", " ".join(rows))
```

```
360
361 if __name__ == "__main__":
362     for fn in (problem_2_5, problem_2_6, problem_2_7):
363         fn()
364         print("done", fn.__name__)
365     (OUT / "results.tex").write_text("\n".join(R) + "\n")
```